

Research Seminar September

Restore the ORAN connection

Oran network wasnt used for a long time (for a month or two) so the container was stopped

Had to restart again

See [Weekly_documentaion/oran-stack-startup.md](#)

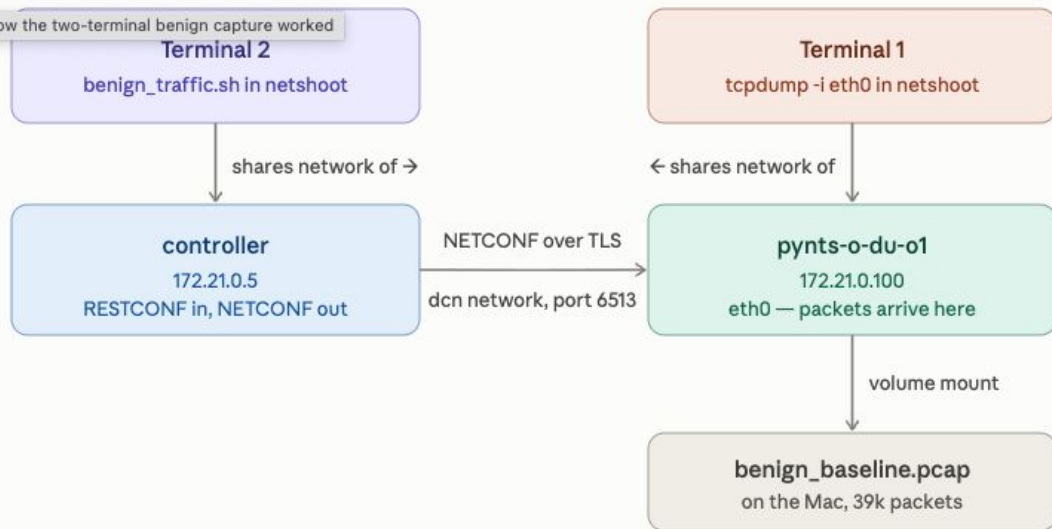
Applying z-score :: Created benign traffic

- Traffic stored in /tmp/benign_active.pcap
- All the steps documented in 'Weekly_documentaion/TLS-benign-capture.md'
- Z-score formula

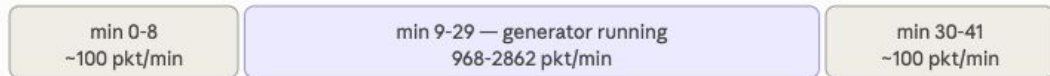
$$z = \frac{x - \mu}{\sigma}$$

- x = The raw data point or observed value
- μ = The population mean (average)
- σ = The population standard deviation

How the two-terminal benign capture worked



What the timeline showed



tcpdump ran the whole 41 minutes; the generator only the middle 20.



Stats of all the protocols

```
F=/tmp/benign_active.pcap
```

```
for P in ip ipv6 tcp udp icmp icmpv6 arp http; do
```

```
    printf "%-8s %s\n" "$P" "$(tshark -r $F -Y $P 2>/dev/null | wc -l)"
```

```
done
```

```
printf "%-8s %s\n" "tls" "$(tshark -r $F -d tcp.port==6513,tls -Y tls 2>/dev/null | wc -l)"
```

```
printf "%-8s %s\n" "tls_hs" "$(tshark -r $F -d tcp.port==6513,tls -Y "tls.handshake.type==1" 2>/dev/null | wc -l)"
```

```
printf "%-8s %s\n" "tcp_syn" "$(tshark -r $F -Y "tcp.flags.syn==1 && tcp.flags.ack==0" 2>/dev/null | wc -l)"
```

- No udp packets

```
ip      33536
ipv6    1866
tcp     35335
udp      0
icmp    64
icmpv6  3
arp      0
http    266
root@wazuh:/# printf "%-8s %s\n" "tls" "$(tshark -r $F -d tcp.port==6513,tls -Y tls 2>/dev/null | wc -l)"
tls     26573
root@wazuh:/# printf "%-8s %s\n" "tls_hs" "$(tshark -r $F -d tcp.port==6513,tls -Y "tls.handshake.type==1" 2>/dev/null | wc -l)"
tls_hs  0
root@wazuh:/#
root@wazuh:/# printf "%-8s %s\n" "tcp_syn" "$(tshark -r $F -Y "tcp.flags.syn==1 && tcp.flags.ack==0" 2>/dev/null | wc -l)"
tcp_syn 133
root@wazuh:/#
root@wazuh:/#
```

Step 2. Window Featuring

1. Featuring windows of benign

...

```
python3 /tmp/extract_windows.py /tmp/benign_active.pcap /tmp/extract_windows/win_benign.csv benign
```

...

```
root@wazuh:/# rm -f /tmp/win_benign.csv
root@wazuh:/# python3 /tmp/extract_windows.py /tmp/benign_active.pcap /tmp/extract_windows/win_benign.csv benign
DONE: 35402 packets -> 1136 windows -> /tmp/extract_windows/win_benign.csv
root@wazuh:/# ls /tmp/extract_windows/
```

- 2.

2. Featuring windows of TLS attack packets

...

```
python3 /tmp/extract_windows.py /tmp/attack_capture/tls_attack.pcap  
/tmp/extract_windows/win_tls.csv  tls_flood
```

...

```
icmp_flood.pcap ping_of_death.pcap tls_attack.pcap tls_flood_2.pcap udp_flood.pcap  
root@wazuh:/tmp# python3 /tmp/extract_windows.py /tmp/attack_capture/tls_attack.pcap /tmp/extract_windows/win_tls.csv tls_flood  
DONE: 16030 packets -> 93 windows -> /tmp/extract_windows/win_tls.csv  
root@wazuh:/tmp#
```

3. Featuring windows of ICMP attack packets

...

```
python3 /tmp/extract_windows.py /tmp/attack_capture/icmp_flood.pcap  
/tmp/extract_windows/win_icmp.csv icmp_flood
```

...

```
icmp_flood.pcap ping_of_death.pcap cts_flood_2.pcap udp_flood.pcap  
root@wazuh:/# python3 /tmp/extract_windows.py /tmp/attack_capture/icmp_flood.pcap /tmp/extract_windows/win_icmp.csv icmp_flood  
read 200000  
read 400000  
read 600000  
read 800000  
read 1000000  
DONE: 1003917 packets -> 32 windows -> /tmp/extract_windows/win_icmp.csv  
root@wazuh:/#
```

Study the csv file

TLS

```
head -3 /tmp/extract_windows/win_tls.csv
```

Feature	Benign (per second)	TLS attack, window 0
<code>syn_count</code>	~0.12	114
<code>tls_handshake</code>	0	37
<code>unique_dports</code>	few	22
<code>pkts</code>	tens	297

```
head -3 /tmp/extract_windows/win_icmp.csv
```

Feature	Benign (per second)	ICMP attack, window 0
icmp_frac	~0	0.9998
pkts	tens	31,364
std_size	high (varied replies)	14.87 → 0 in steady windows
syn_count	~0.12	0
tls_handshake	0	0

What about UDP attacks

- On O1 interface, there is no UDP based communication
- Controller talking to the O-DU — only ever produces NETCONF over TLS, because that's what O1 management is
- How the UDP attack is handled instead Suricata rule detects the UDP flood by threshold

Step3 : Build the model

a) Train on the first 80%, hold back the last 20% to measure false positives:

Script written 'splitting_benign_data.py'

```
root@wazuh:/tmp/extract_windows# mv splitting_data.py splitting_benign_data.py
root@wazuh:/tmp/extract_windows# python3 splitting_benign_data.py
train 908, test 228
root@wazuh:/tmp/extract_windows#
```

- This created benign_train.csv and benign_test.csv file

b) Test the score using anomaly detector – z_score_model.py

This trains on benign_train, then scores benign_test, win_tls, and win_icmp

```
root@wazuh:/tmp/extract_windows# nano z_score_model.py
root@wazuh:/tmp/extract_windows# python3 z_score_model.py
benign_test  15/228 false positives (6.6%)
tls_flood    66/93 detected (71.0%)
icmp_flood   32/32 detected (100.0%)
root@wazuh:/tmp/extract_windows#
```

Isolation Forest

Save the script in isolation_forest.py

Isolation forest vs z-score

```
python3: can't open file 'extract_windows/isolation_forest.py':  
[Errno 2] No such file or directory  
root@wazuh:/tmp/extract_windows# python3 isolation_forest.py  
benign_test 1/228 false positives (0.4%)  
tls_flood   1/93 detected (1.1%)  
icmp_flood  32/32 detected (100.0%)  
root@wazuh:/tmp/extract_windows#
```

Stop till zscore

present methodology, strategies

The architecture should be base on odu and smo,